

DBMS-CT2

① Define hash fn and its examples.

Ans: A hash fn is a mathematical fn that converts input data (called a key) into a fixed size value, hash code, or hash address.

[# Main purpose → make data access fast.]

A hash fn maps - Key → Hash Address

if roll-no = 123 : $h(123) = 123 \% 10 = 3$ → Address

characteristics of good hash fn - should

- Be fast to compute
- Distribute data uniformly
- Minimize collisions
- Produce consistent o/p for same i/p.

Types

Modulo hash fn

$$h(k) = k \% m$$

key Table size

$$h(57) = 57 \% 10 = 7 \checkmark$$

Folding Method

$$h(123456) = 123 + 456 = 579$$

Mid Sq. Method

$$h(25) = 25^2 = 625$$

take mid digit → 2

Simple hash function using modulo method

```
def hash_function(key, table_size):
    return key % table_size
```

Example keys

```
keys = [23, 45, 12, 67, 34]
```

```
table_size = 10
```

```
for key in keys:
```

```
    index = hash_function(key, table_size)
    print(f"Key {key} → Hash Index {index}")
```

► Output -

```
Key 23 → Hash Index 3
Key 45 → Hash Index 5
Key 12 → Hash Index 2
Key 67 → Hash Index 7
Key 34 → Hash Index 4
```

Q What is Relational Algebra & Rel. Calculus?

{ Procedural query language }

{ non-procedural query language }

it specifies:

{ what operatn to perform }

{ how to perform it }

Basic Operations of Relational Algebra

Operation	Symbol	Purpose
Selection	σ	Select rows
Projection	π	Select columns
Union	$\cup$	Combine relations
Difference	$-$	Find difference
Cartesian Product	$\times$	Combine tuples
Rename	ρ	Rename relation

Eg. Table Student.

Roll	Name	dept
1	A	CSE
2	B	IT
3	C	CSE

$\sigma_{\text{Dept} = \text{CSE}} (\text{STUDENT})$ → only rows having CSE

↳ TN

Relational Calculus: specifies — what data is required.
— not how to retrieve it.

TRC
[Tuple Relational Calculus]
variables represent entire tuples (rows) of a table.

DRC

[Domain Relational Calculus]
variables represent attribute values (columns/domains) instead of whole tuples.

$\{t \mid \text{STUDENT}(t) \wedge t.\text{Dept} = \text{CSE}\}$

↓
Select all student tuples where dept. is CSE.

$\{ \langle \text{Name} \rangle \mid \exists \text{Roll, Dept} (\text{STUDENT}(\text{Roll}, \text{Name}, \text{Dept}) \wedge \text{Dept} = \text{CSE}) \}$

↓
Display names of students whose dept. is CSE.

Q3. Sequential File Organization in DB?

Simplest type of File Organization, where files are stored one after the other, rather than storing different files in rows and cols (in tabular form), storing data in rows.

Complexity while searching data: $O(N)$

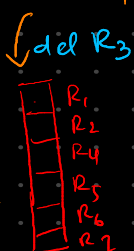
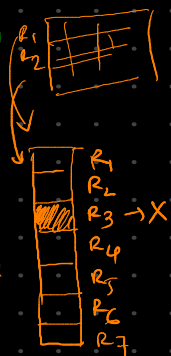
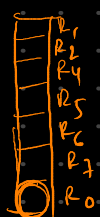
File organization

describes the conn. b/w records that make a file. It is the practice of storing info in order.

Methods

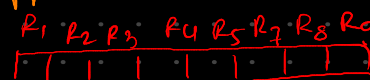
File Method

- Blocks are stored successively
- Data entered in the same order in which things are in the table.
- For deletion: delete the row.
- Insertion: at the end.

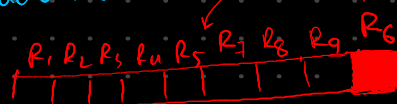


Sorted File Method

- A new file are always added in the last
- After adding the whole sequence is sorted.
- Eg: Suppose below are the sorted files.



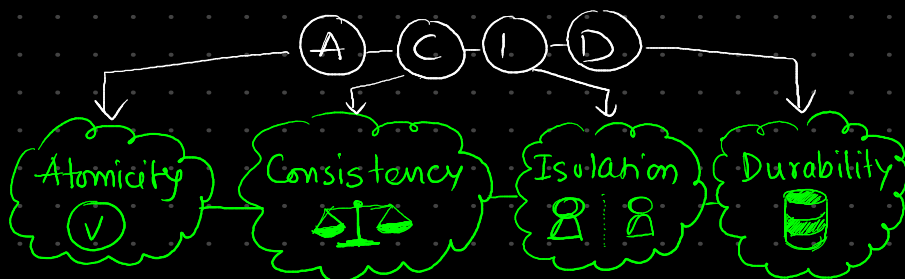
add R6



sort.



Q4. ACID properties of DBMS.
Set of Rules that ensure reliable and consistent transaction processing in DBMS.



Atomicity

- A transaction is treated in a single unit
- Either all operations are completed or None are completed.
- Eg: ₹1000 transferred from A → B A: ₹1000 debited
B: ₹1000 credited
- If this fails all the operations roll back.

Consistency

- Ensures that the database remains in a valid state before and after the transaction.
- Maintain database Rules and constraints.
- Before Transaction

Before Transaction

A = ₹5000
B = ₹3000
<u>total = ₹8000</u>

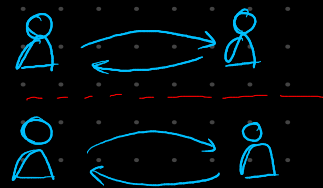
After Transaction

A = ₹4000
B = ₹4000
<u>total = ₹8000</u>

total Remains Same

Isolation

- Multiple transaction executes independently without interfering with each other.
- Intermediate changes of one transaction are not visible to others.
- 2 users updating the same bank account simultaneously should not create incorrect balance values.



Durability

- Once the transaction is committed the results should be permanently stored in the database.
- Even if power failure or crash occurs - data remains safe.

Q.5 Define functional dependency (FD)

FD is a relationship between 2 attributes in a relation (table).

It states that: One attribute uniquely determines another attribute.

$A \rightarrow B$ → A determines B

Types

① Full Functional Dependency

An attribute depends on the whole pk.

② Partial Dependency: An attribute depends only on part of a composite key.

(Student ID, Course ID) → Student Name
depends only on Student ID.

(Student ID, Course ID) → Grade
grade depends on both Student ID & course ID.

③ Transitive Dependency one non-key attribute depend on another non-key attribute.

Student ID $\rightarrow$ Dept ID
 Dept ID $\rightarrow$ Dept Name } This is transitive dependency.

Q.6. Define Normalization $\rightarrow$ process of organizing data in a db to reduce redundancy and improve data integrity.

Normalizⁿ is a db design technique used to eliminate -

Objective of Normalizⁿ

1. Minimize duplicate data
2. Improve data consistency
3. Reduce Anomalies
4. Organize db efficiently.

- Data Redundancy
- Insertion Anomalies
- Update Anomalies
- Deletion Anomalies.

Student

ID	Name	Course	Faculty
1	A	DBMS	B
1	A	OS	C

Normalization $\rightarrow$

Student

ID	Name
1	A

Course

Course	Faculty
DBMS	B
OS	C

- Name Repeated
- Wastage of storage
- Difficult to update.

Normal Form

- 1NF First Normal Form
- 2NF 2nd Normal Form
- 3NF 3rd Normal form

- No Repeating groups
- Atomic values only
- Must be in 1NF
- No partial dependency
- Must be in 2NF
- No transitive dependency.

Disadvantages of Normalizⁿ

- More tables are created.
- Complex queries using joins.
- Slightly slower retrieval in some cases.

Q7. what is Index and how does it improves performance?

INDEX $\rightarrow$ is a data structure used to speed up data retrieval operations from a db table.

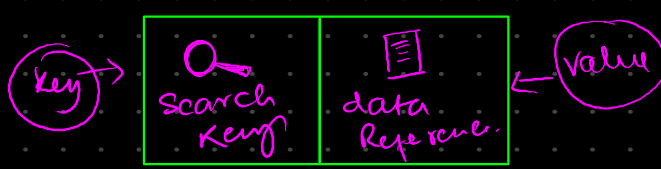
An INDEX is a special lookup table that stores:

- Indexed attribute values.
- Corresponding record addresses.

Attributes of Indexing.

- ① Access Type: type of access, value based, Range etc.
- ② Access Time: time needed to find a particular data element.
- ③ Insertⁿ Time: time taken to find space and insert.
- ④ Deletion Time: time taken to find & delete.
- ⑤ Space Overhead: Additional space Required by index.

Structure of an Index.



- Types**
- ① **Primary Index**: Created on PK. Eg: Roll-no.
 - ② **Secondary Index**: Created on Non-primary attributes: Name, Dept.
 - ③ **Clustered Index**: Stores records physically in sorted order.
 - ④ **Non Clustered Index**: Stores separate pointers to records.

Advantages of INDEX 😊

- Faster Searching
- Faster Sorting
- Improves query performance
- Efficient data Retrieval.

Disadvantages 😞

- Requires extra storage
- Update operations become slower.
- INDEX Maintenance Req.

Q.8. What is Stored Procedures in DBMS?

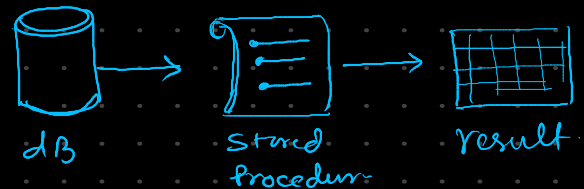
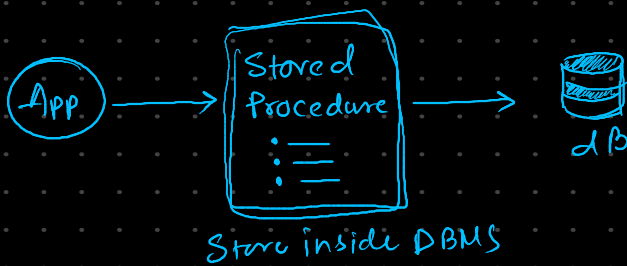
✓ Set of SQL Queries saved in the DBMS that can be executed whenever needed by calling its name.

Suppose we want to display all students from the CSE department repeatedly.

```

<> SQL
CREATE PROCEDURE ShowCSEStudents
AS
BEGIN
    SELECT * FROM STUDENT
    WHERE Dept = 'CSE';
END;
  
```

Procedure Name



Advantages of Stored Procedure

- Reduce repeated SQL code
- Faster Execution
- Improves Security
- Easy Maintenance
- Reduces Network traffic.

Disadvantages

- Debugging can be difficult
- Consumes DB memory
- DB dependent.

Q.9 What is a **VIEW** in DBMS and what are its Advantages?

View is a logical or virtual table generated from the result of a SQL query.

Original STUDENT Table

Roll_No	Name	Dept	Marks
101	Anik	CSE	90
102	Rahul	ECE	85

Creating a View

<> SQL

```
CREATE VIEW Student_View AS
SELECT Name, Dept
FROM STUDENT;
```

Using the View

<> SQL

SELECT * FROM Student_View;

Result:

Name	Dept
Anik	CSE
Rahul	ECE

characteristics of VIEW

- Virtual table
- Created From SQL Query
- Does not store Actual data separately
- Simplifies complex Queries.

Advantages

- Improves Security - users can access limited data only.
- Simplifies Complex Queries - complex joins and queries can be hidden inside a view.
- Data Abstraction - User do not need to know full table structure.
- Easy data access - provides customized data representation.

Q.10 What is DBMS Schema?

→ overall logical structure or Blueprint of a DB.
It defines tables, attributes, relationships, constraints and data types.

Suppose A Student table.

Roll_No	Name	Dept
101	Anik	CSE
---	---	---

Schema can be written as:

STUDENT (Roll no., Name, Dept)

↑
Table Name↓
AttributesTypes of Schema① External:

- User level view
- Different users see diff't data acc. to their needs.
(Eg: Students see Marks only)

② Conceptual Schema

- Logical Structure of the whole db.
- Define relationships among tables.

③ Physical:

- Describe how data is physically stored.
- Includes files, indexes, storage method etc.

3 Schemas Architecture

USERS/ VIEWS

— External Level

CONCEPTUAL SCHEMA

— Conceptual Level

PHYSICAL SCHEMA

— Physical Level

